МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №4

Тема: **«Структурные паттерны: Компоновщик, Декоратор»**

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: реализовать решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов из списка: «Компоновщик», «Декоратор».

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Компоновщик

Где находится: coin_collector/multiplayer.py, класс PlayerComposite.

Задача: одинаково обновлять и отрисовывать всех участников матча, не дублируя циклы для одного и нескольких игроков.

```
class PlayerComposite:
    """Apply shared game operations to all players in a match."""

    def __init__(self, players: list[Player]) -> None:
        if not players:
            raise ValueError("A player composite must contain at least
one player")
        self.players = tuple(players)

    def update(
        self,
        frames: tuple[InputFrame, ...],
        delta_time: float,
        bounds: pygame.Rect,
    ) -> None:
        if len(frames) != len(self.players):
            raise ValueError("Each player must have exactly one input
frame")
        for player, frame in zip(self.players, frames):
            player.update(frame, delta_time, bounds)

    def draw(self, drawing: DrawingAPI) -> None:
        for player in self.players:
            player.draw(drawing)
```

Как работает: компоновщик хранит игроков и применяет к каждому соответствующий кадр ввода. Сам TwoPlayerMatch обращается к компоновщику так же, как к общей группе игроков.

```
players = PlayerComposite(level.players)
players.update((player_one_frame, player_two_frame), delta_time,
level.bounds)
players.draw(drawing)
```

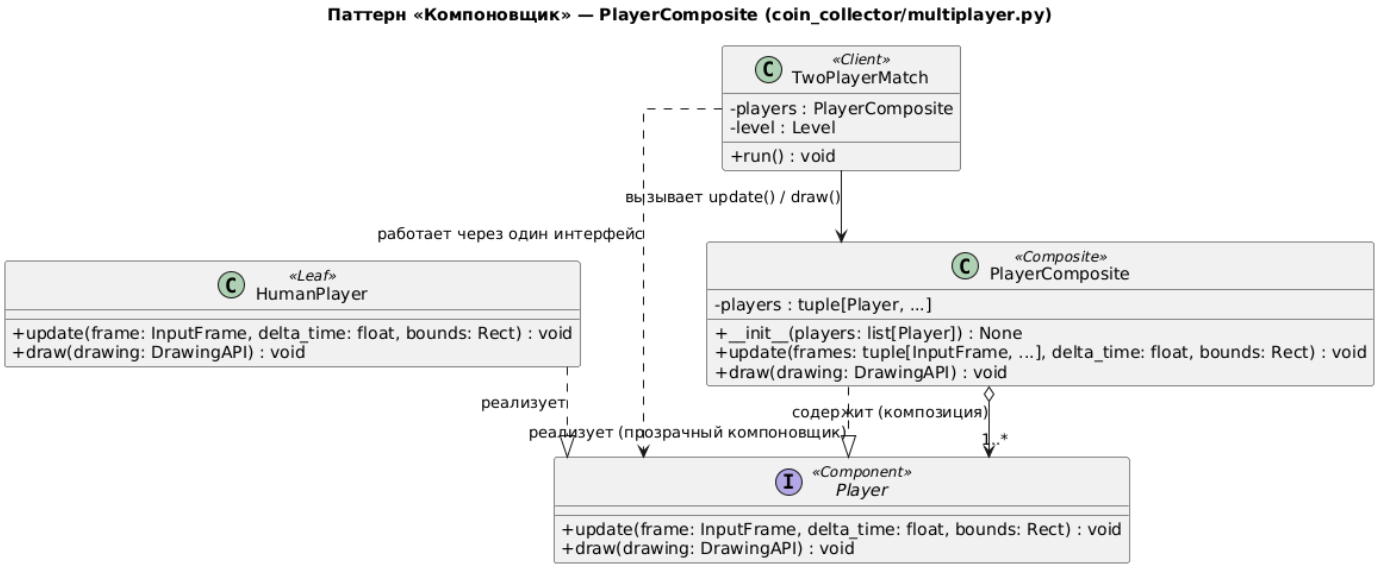


Рисунок 1. Диаграмма паттерна "Компоновщик"

Таким образом, общее движение и отрисовка масштабируются на группу участников, сохраняя индивидуальное управление каждым игроком.

2. Декоратор

Где находится: coin_collector/entities.py, класс BonusCoin; бонусные монеты добавляет GameLevelBuilder.add_bonus_coin().

Задача: добавить обычной монете дополнительную ценность и заметный внешний вид без изменения её базовой реализации.

```
class BonusCoin:
    """Decorate a coin with a higher value and a visible bonus ring."""

    def __init__(self, coin: Coin, multiplier: int = 2) -> None:
        if multiplier < 2:
            raise ValueError("A bonus coin multiplier must be at least
2")
        self._coin = coin
        self.value = coin.value * multiplier

    @property
    def rect(self) -> pygame.Rect:
        return self._coin.rect

    def update(self, delta_time: float) -> None:
        self._coin.update(delta_time)

    def draw(self, drawing: DrawingAPI) -> None:
        drawing.ellipse((110, 222, 255), self.rect.inflate(12, 12))
        self._coin.draw(drawing)
```

Как работает: BonusCoin оборачивает Coin, предоставляет тот же интерфейс Collectible, делегирует обновление и отрисовку и удваивает начисляемые очки. При отрисовке он добавляет голубое кольцо.

```
bonus = BonusCoin(Coin((480, 360)))
assert bonus.value == 20
bonus.update(0.016)  # обновляет обёрнутую монету
```

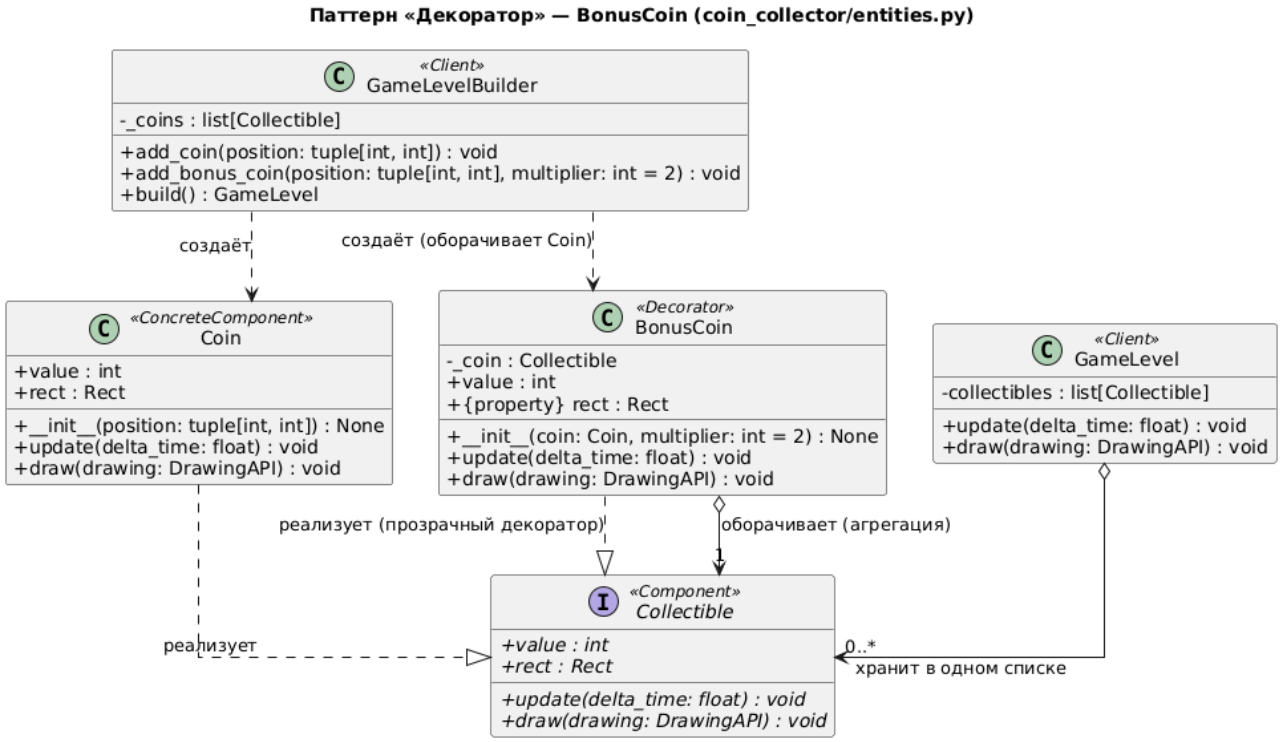


Рисунок 1. Диаграмма паттерна "Фасад"

Таким образом, обычные и бонусные монеты могут находиться в одном списке уровня и обрабатываются одним циклом сбора.

Вывод:

Проделав работу, реализовал решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов из списка: «Компоновщик», «Декоратор».